An Accurate Instruction-Level Energy Estimation Model and Tool for Embedded Systems

Mostafa Bazzaz, Mohammad Salehi and Alireza Ejlali

***Abstract*—Estimating the energy consumption of applications is a key aspect in optimizing embedded systems energy consumption. This paper proposes a simple yet accurate instruction-level energy estimation model for embedded systems. As a case study, the model parameters were determined for a commonly used ARM7TDMI-based microcontroller. The total energy includes the energy consumption of the processor core, Flash memory, memory controller, and SRAM. The model parameters are instructions opcode, number of shift operations, register bank bit flips, instructions weight and their Hamming distance, and different types of memory accesses. Also, the effect of pipeline stalls have been considered. In order to validate the proposed model, a physical hardware platform equipped with energy measurement capabilities was developed. We have conducted experiments on several embedded applications from MiBench benchmark suite and the results show less than 6% error in the energy consumption estimation. We have also developed an energy profiler tool for the systems that use ARM7TDMI processors by embedding the model parameters in an instruction-level profiler from the SimpleScalar toolset which provides valuable information and guidelines for software energy optimization.**

***Index Terms*—Embedded software, energy estimation, energy measurement, memory, regression analysis.**

I. Introduction

EMBEDDED systems play an important role in many areas of human life. Cellphones, PDAs, and satellites are only few examples of devices with a processor embedded in them. A large group of these systems are portable battery-powered devices that have a limited source of energy. This makes the energy consumption a prominent characteristic. Hence, energy estimation during design phase of these applications helps designers in optimizing the energy consumption and the battery lifetime. Since software is responsible for a large portion of the system energy consumption, an accurate energy model is necessary for the system energy optimization [1]. There are two main models of embedded software energy estimation: measurement-based and simulation-based.

In the simulation-based approach a simulation model of the target hardware is used to run the applications and calculate the energy consumption of each part of the system which may be as detailed as gate level such as the model presented by Klass *et al.* [2] or as abstract as behavioral level like the model presented by Callou *et al.* [3]. This approach needs the simulation model of all hardware modules that are mostly unavailable or very expensive. Also, evaluating the impact of a small change in an instruction opcode or operands on the total energy consumption of the system requires rerunning the simulation.

Measurement-based methods use data obtained from a physical target device. Most of these models, like those presented by Chang *et al.* [4], Nikolaidis *et al.* [5], and Konstantakos *et al.* [6], associate the instructions with the corresponding energy cost. The total application energy consumption is the aggregate cost of all executed instructions that can be calculated by running the application in an emulator. The main advantage of measurement-based methods is high accuracy in the energy estimation due to the real values obtained from the target platform.

It should be noted that for most of the commercially used microcontrollers (e.g., AT91SAM7X256 considered in this paper) there is no authentic SPICE (or any other detailed) simulation model; hence, it is not possible to adopt a simulation-based approach. Furthermore, we believe that as measurement-based approaches use physical and real systems, they are always more accurate than simulation-based approaches. Hence, in this paper we adopt a measurement-based approach.

Energy estimation models are also categorized according to their scope. The models presented in [4], [5], and [7] only model the processor core and the model presented in [6] models an embedded system including a microcontroller, external RAM, and external A/D converter.

In this paper, we introduce a new instruction-level energy estimation model and tool for an ARM7TDMI-based microcontroller including the processor core, internal Flash and SRAM memories. While our model is simple enough to be implemented in any instruction set simulator, it provides good accuracy and can be used to estimate the energy consumption of the processor core (ARM7TDMI), SRAM, and Flash memory units of an embedded system. Also, the process of deriving the model allows for easy recalibration of the model for other platforms. The model is based on parameters such as instructions type, number of executed shift operations, register bank bit flips, weight and Hamming distance of the instruction words, and it has been validated using the MiBench benchmark suite [8]. Also, different type of memory accesses and pipeline stalls have been considered.

Manuscript received February 22, 2012; revised September 12, 2012; accepted October 24, 2012. Date of publication March 12, 2013; date of current version June 5, 2013. The Associate Editor coordinating the review process was J. Sheppard.

The authors are with the Department of Computer Engineering, Sharif University of Technology, Tehran, 11365-11155, Iran (e-mail: bazzaz,mohammad_salehi@ce.sharif.edu; ejlali@sharif.edu).

Color versions of one or more of the figures in this paper are available online at http://ieeexplore.ieee.org.

Digital Object Identifier 10.1109/TIM.2013.2248288

0018–9456/$31.00 © 2013 IEEE

Compared with previous methods, our model:

1) provides equal or better accuracy as compared to [3], [7], and [9]. Note that this comparison might not be applicable to other previous works. For example, the model presented by Zotos *et al.* [10] analyzes the growth of energy consumption and not the exact figures of power and energy, and hence, this comparison is not applicable to [10]. Also, some research works consider only a single application (e.g., the works of Steinke *et al.* [11] and Konstantakos *et al.* [6], [12]), and hence, although their results may be accurate for that specific application, their conclusions cannot be extended to other applications;
2) does not need cycle-accurate simulation that improves the simulation speed (e.g., unlike the model presented by Change *et al.* [4]);
3) is validated by a physical hardware implementation using MiBench [8] benchmarks that are considered as representatives for embedded applications (unlike the works presented in [3], [4], [7], [11], [12], and [6]);
4) proposes a simpler model for estimating the interinstruction energy consumption that improves the simulation speed of large workloads (as compared to works presented in [4], [7], [9], [11]). For example, in the model presented by Steinke [11] we need to calculate an equation with 12–32 parameters for each instruction, while our technique involves only 4–6 parameters when calculating the energy consumption of each instruction. While we have reduced the number of parameters, the accuracy is not compromised. This is achieved by considering a single parameter for the activity of the register file (unlike the works presented in [4], [6], [7], and [12]) and a separate parameter for Shift operations (unlike the works presented in [4]–[7], [11], and [12]);
5) is suitable for a wide range of applications (unlike application specific estimation models such as the model presented by Konstantakos *et al.* [6], [12]);
6) considers the energy consumption of Flash memory (unlike the works presented in [4], [5], [7], [9], [11], and [6]) and SRAM (unlike the works presented in [4], [5], [7], and [9]). Those previous works that consider memory units usually report the energy consumption of memory units as a whole. However, in our proposed model, the energy consumptions of SRAM and FLASH units have been considered separately, enabling designers to analyze and optimize the energy consumption of each unit individually.

The rest of the paper is organized as follows. Section II reviews the related works. In Section III our energy measurement method and hardware platform architecture are explained in detail. In Section IV the proposed energy model is demonstrated. Section V presents the derivation process of our model. The results of the experiments are discussed in Sections VI, and section VII concludes the paper.

II. Related Works

Tiwari *et al.* [1] have presented an energy modeling of embedded softwares executed by an embedded processor. This model includes only the energy of instructions and interinstruction energy costs (energy consumption caused by sequential execution of two different instructions).

Chang *et al.* [4] have analyzed the energy consumption of a modified ARM7TDMI core prototype and have proposed an energy estimation model based on the opcode of instructions, number and value of registers, instruction and data fetch addresses, and the immediate operands in each stage of the pipeline. In this paper, an equation in the form of $\alpha h + \beta w + \gamma$ has been derived for each stage of the pipeline where h is the Hamming distance between the current and previous values and w is the weight of the current value. α, β, and γ are instruction dependent constants that are measured for each instruction type. The total energy is calculated as the sum of the energy consumption of each instruction throughout the pipeline stages. However, this model does not consider the energy consumption of the Flash memory or SRAM.

Nikolaidis *et al.* [5] have studied the drawn current of pipelined processors during instruction execution and have concluded that the energy consumption of an instruction affects the measured current at different clock cycles. In this paper, a model for in-order pipelined processors based on the model proposed by Chang *et al.* [4] with the inclusion of pipeline stall energy costs has been proposed. The final model does not include the energy consumption of the Flash memory and SRAM.

Kavvadias *et al.* [9] have improved the energy estimation model proposed by Nikolaidis *et al.* [5] by including the pipeline flush and pipeline stall caused by load store, multiply or branch instructions. The final model is evaluated for an ARM7 processor using real software kernels. However, the energy consumption of memories has not been considered in these works.

Zotos *et al.* [10], instead of addressing the system (memory) energy consumption in terms of energy values (numbers),attempt to investigate how the system (memory) energy consumption grows as the size of the input data increases. This is indeed very similar to the way people analyze the time-complexity of algorithms where the exact and explicit values of execution time is not considered and only the execution-time growth is analyzed.

Steinke [11] targets an ARM7-based platform and proposes a more detailed energy model. The total energy consumption is modeled as the sum of instruction-dependant CPU and memory energy consumption plus the activation cost of each functional unit of the processor. This model includes the same parameters as the model proposed by Kavvadias *et al.* [9] as well as the Hamming distance and weight of the register values, address bus, data bus, and the activation cost of each functional unit. The model does not cover the energy consumption of pipeline stalls or the Flash memory, and it is only tested for a sequence of 12 instructions. Note that instructions like Branch and Load have different impacts on energy consumption from simpler instructions like ADD or SUB, and hence, 12 random instructions might not be considered as a representative of real world complex applications.

Konstantakos *et al.* [6] have modeled the energy consumption of a system comprising a Motorola HC908GP32

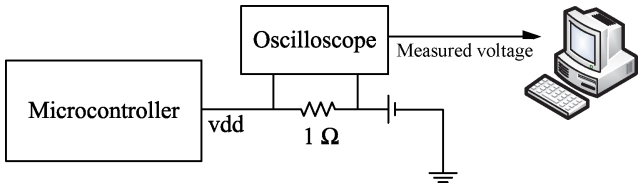

Fig. 1. Architecture of our measurement system.

microcontroller, an external A/D converter and an external DRAM. The energy consumption of each module is measured separately. Unlike the other models, the energy consumption of the core is estimated based on the number of execution cycles of each instruction rather than the type of the instruction. In this way the number of parameters is significantly reduced that results in a simpler model. But the model is only tested for a single benchmark that is the logging application, and various benchmarks have not been considered. Also, the model does not consider the energy consumption of Flash memory.

Most of the mentioned studies calculate the model coefficients by running sequences of instructions and measuring the energy consumption of the system during the execution cycles of each instruction. But Lee *et al.* [7] have used a "black-box" or "stimulus-response" approach where a set of test programs are executed on the hardware platform, the energy consumption is measured, and the model parameters are extracted using the regression methods. As this approach does not require detailed information about the internal structure of the processor, it improves the model retargetability. This model is similar to [4] and it is tested by executing randomly generated instructions on an ARM7TDMI core. It only estimates the energy consumption of the processor and does not cover the energy consumption of memories or peripherals.

In this paper, a new simplified version of the processor energy estimation model based on the models presented by [9], [11], and [7] has been introduced. The parameters of this simplified model do not require cycle-accurate simulation. The energy consumption of memories is also modeled similar to the model presented by Steinke *et al.* [11] but with a smaller number of parameters. For hardware energy measurement a stimulus-response approach similar to [7] has been used that will be explained later in Section IV.

III. Measurement Method and System Architecture

Precise measurement of the energy is one of the most important challenges in utilizing measurement-based methods. Some measurement-based studies such as the works of Lee *et al.* [13], Tiwari *et al.* [1], and Russell [14] calculate the energy consumption by reading the average current drawn by the system from the power supply. We have used a similar approach by placing a 1-ohm resistor at the power supply pin of the microcontroller. The measured current (which is read by a high frequency oscilloscope) and the application execution time are sent to the host computer for calculating the energy consumption. Fig. 1 illustrates this architecture.

In this paper, an AT91SAM7X256 microcontroller [15] (based on ARM7TDMI processor core [16]) is used as the target platform. The internal structure of this microcontroller is shown in Fig. 2. As we only concentrate on the energy consumption of the processor core, SRAM and Flash, all other modules are disabled by setting the appropriate flags in the initialization part of the program source code. The Flash memory is used for storing the code and read-only data while the SRAM is utilized as runtime data memory.

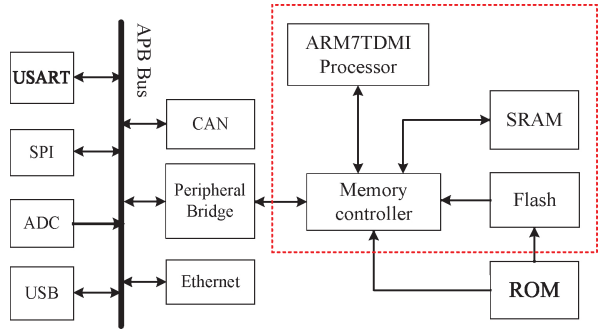

Fig. 2. Microcontroller internal structure. Our proposed model estimates the energy consumption of the CPU, Flash memory, SRAM, and memory controller.

The ARM7TDMI core is a 32-bit RISC microprocessor specialized for low-power applications and it is capable of achieving very high MIPS per watt with a three-stage pipeline and no cache memory. The pipeline stages include instruction fetch (IF), decode (ID), and execute (EX). ARM7TDMI employs 34 different assembly instructions where five of them are coprocessor communication instructions. However, in this paper, we do not intend to consider the coprocessor unit, and hence, we do not consider these instructions.

IV. Proposed Energy Consumption Model

Most software programs that run on embedded processors consist of two parts.

1) An initialization part that configures system modules, initializes program variables, etc. This part of the program is executed only once at the start of the program in order that the system gets ready to perform its main operation.
2) The main part that is usually implemented as an endless loop.

From an energy consumption viewpoint, we can ignore the initialization part and assume that the system always operates in its main part. This is because when we turn an embedded system on, it is in the initialization phase for only few micro seconds and then it goes into the main phase where it operates for hours. This implies that almost all the energy consumption of an embedded system is because of the main phase and not the initialization phase. Based on this assumption, in this paper, we do not consider applications where there is no differentiation between initialization and main parts, e.g., systems with very small programs that their execution takes only a few instruction cycles.

We can express the total energy consumption of each instruction as the sum of fetch energy (E_{fetch}), decode energy

(E_{decode}), execute energy (E_{execute}), and static energy (E_{static}), where

$$\begin{aligned} E_{\text{fetch}} &= E_{\text{cntr(code)}} + E_{\text{Flash(code)}} + E_{IF} \\ E_{\text{decode}} &= E_{ID} \\ E_{\text{execute}} &= E_{EX} + E_{\text{cntr(data)}} + E_{\text{Flash(data)}} + E_{\text{SRAM}} + E_{\text{stall}} \end{aligned} \quad (1)$$

($E_{\text{cntr(code)}}$ is the amount of energy consumed in the memory controller caused by the code, $E_{\text{cntr(data)}}$ is the amount of energy consumed in the memory controller caused by the data, $E_{\text{Flash(code)}}$ is the amount of energy consumed in the Flash memory caused by the code and $E_{\text{Flash(data)}}$ is the amount of energy consumed in the Flash memory caused by the data.) Since we have disabled all modules except the core, memories and the memory controller, there must be no instruction that causes energy consumption in the peripherals. However, the microcontroller datasheet does not clearly specify how the peripherals are disabled. If their power supply lines are completely cut off, then there will be no E_{static}. But if the supply lines are not cut off, and hence, the circuitry is turned on but inactive, they may consume a small amount of static power, which means E_{static} will not be zero.

We separate the energy consumed for reading and processing (executing) an instruction from the energy consumption of data transfer, and calculate the energy consumption of an instruction as the sum of process energy (E_{process}), memory energy (E_{memory}) and static energy (E_{static}), where

$$\begin{aligned} E_{\text{process}} &= E_{cntr(\text{code})} + E_{Flash(\text{code})} + E_{IF} \\ &\quad + E_{ID} + E_{EX} + E_{\text{stall}} \end{aligned} \quad (2)$$

$$E_{\text{memory}} = E_{\text{cntr(data)}} + E_{Flash(\text{data})} + E_{\text{SRAM}} \quad (3)$$

E_{memory} only depends on the number of memory accesses (for data transfer) during the program execution. Note that calculating some of these energy segments needs cycle-accurate simulation of the program, however, we estimate them using other parameters that can be calculated during instruction-level simulations.

In the following, we introduce the energy consumption models that are used for modeling the total energy consumption of the system.

A. Memory Energy Consumption Model

Generally, the energy cost of accessing the RAM can be modeled with the number of read and written bytes. Also, the static energy consumption of the SRAM is considered as a part of E_{static}. Therefore we have

$$\begin{aligned} E_{\text{SRAM}} &= N_{\text{SRAM(read)}} \times \lambda_{\text{SRAM(read)}} \\ &\quad + N_{\text{SRAM(write)}} \times \lambda_{\text{SRAM(write)}} \end{aligned} \quad (4)$$

where $N_{\text{SRAM(read)}}$ is the number of read bytes, $N_{\text{SRAM(write)}}$ is the number of written bytes, and $\lambda_{\text{SRAM(read)}}$ and $\lambda_{\text{SRAM(write)}}$ are the energy cost per byte transfer (reading or writing one byte).

Analyzing the MiBench benchmark suite [8] applications (Fig. 3) shows that about 30% of the memory accesses target the Flash memory. To analyze the difference between the energy consumption of the Flash memory and SRAM, another experiment was conducted with two sets of test programs (Benchmark 1 and Benchmark 2). These benchmarks are identical, except for the target address space. While the target address space of Benchmark 1 is within the Flash ($0\times100\,000$), the target address space of Benchmark 2 is within the SRAM ($0\times200\,000$). These benchmarks were executed with different target addresses and their energy consumptions were measured. We observed that the average energy consumption of Benchmark 1 is 20% higher than that of Benchmark 2. The results also show that the energy consumption of accesses to different locations of the same memory module (SRAM or Flash) are almost constant. Hence, only the number and type of the memory accesses are considered in the memory energy estimation model and not the target address. Due to the notable share of the Flash memory access in the total application memory accesses (see Fig. 3) and the difference between the energy consumption of Flash memory accesses and SRAM accesses, two separate energy parameters are considered for the Flash and SRAM read accesses in our energy model. In the context of embedded systems, Flash write operations are relatively rare and are usually performed during the offline phase (programming/configuration phase) of an embedded system where system can be connected to electricity power lines and does not use battery. However, during the normal operation of embedded systems where system is battery-operated (and hence energy consumption and energy estimation is prominent), the system usually accesses the Flash by read operations (for example to fetch instructions), and hence, we have to consider Flash read operations in energy estimation.

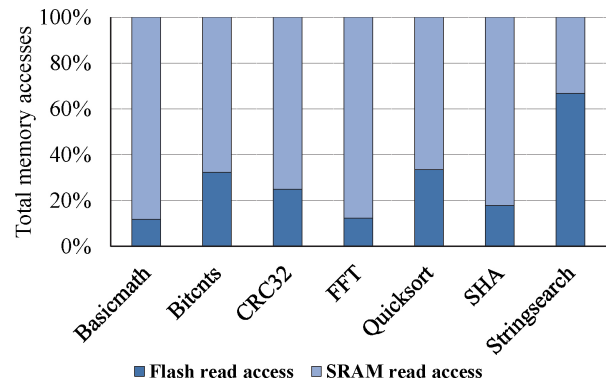

Fig. 3. Read access count of the Flash memory and SRAM in the MiBench benchmark suite applications.

Considering the memory parameters, (3) (the energy consumption of the memory) can be rewritten as

$$\begin{aligned} E_{\text{memory}} &= N_{\text{Flash(read)}} \times \lambda_{\text{Flash(read)}} + N_{\text{SRAM(read)}} \\ &\quad \times \lambda_{\text{SRAM(read)}} + N_{\text{SRAM(write)}} \times \lambda_{\text{SRAM(write)}} \end{aligned} \quad (5)$$

where $N_{\text{Flash(read)}}$ is the number of bytes read from the Flash memory and $\lambda_{\text{Flash(read)}}$ is the energy cost of reading one byte from the Flash memory.

B. Processor Core Energy Consumption Model

The energy consumption of the different pipeline stages are grouped together as the processor core energy consumption

E_{core}. Therefore, (2) can be simplified as

$$E_{\text{process}} = E_{\text{Flash(code)}} + E_{\text{cntr(code)}} + E_{\text{core}} + E_{\text{stall}} \quad (6)$$

Some previous studies have shown that the energy consumption of the processor core during the execution of an instruction can be divided into three parts: *i)* the base energy cost, which only depends on the current instruction, *ii)* the interinstruction cost, which is the amount of energy consumed by the processor during the consecutive execution of different instructions, and *iii)* the pipeline stall cost [4], [5], [7], [9]. Let B_i be the base energy cost of executing the instruction i, $I_{i,j}$ be the interinstruction energy cost of executing instruction j after i, E_{st} be the energy cost of the pipeline stall, and S be the set of all pipeline stalls during the application execution. The processor core energy consumption during the execution of an application with n instructions can be estimated as

$$E_{\text{core}} = \sum_{i=1}^{n}(B_i + I_{i,i-1}) + \sum_{s \in S} E_{st}. \quad (7)$$

According to the previous studies B_i depends on the current instruction type, the operands type, the addressing mode and the operands weight [4], [7]. The $I_{i,j}$ depends on the current and previous instructions type and their operands Hamming distance. In our experimental setup, the program code is read from the Flash memory. Executing each instruction causes a read access to the Flash memory to fetch the instruction code and causes energy consumption in the controller $E_{\text{cntr(code)}}$ and the Flash memory $E_{\text{Flash(code)}}$ (see (2)). Since this portion of energy consumption is present during the execution of each instruction, we do not consider an extra parameter for it in our model and it will be covered as a portion of the base instruction cost B_i.

In most cases, the interinstruction energy cost is about 5% of the base instruction cost [9]. Thus, we simplify the model by ignoring detailed interinstruction cost estimations and use other parameters, considering the internal structure of the processor core [16]. These parameters include the Hamming distance and weight of the instructions, number of bit flips in the register bank, and the number of shift operations. In the following we discuss these parameters in detail.

1) *Instructions Word Hamming Distance:* Any change in the input signals of a circuit can cause a series of activities in the circuit. In some designs these subsequent activities and their corresponding energy consumption have a direct relationship to the changes in the input signals. To exploit this idea in our proposed model, the effect of the instruction Hamming distance on the core power consumption was analyzed by running a series of benchmarks. Each of these benchmarks contains a large amount of two different types of instructions. Benchmark 1 comprises two separate blocks where all instructions of each block are of the same type. Benchmark 2 orders the instructions in an interleaved manner where each instruction of type A is followed by an instruction of type B and vice versa. The energy consumption of Benchmark 2 shows about 6% increase over that of Benchmark 1. We model this portion of energy consumption by simply counting the Hamming distance between the instructions during the execution flow.

2) *Instructions Word Weight:* The energy consumption of an instruction also depends on the operands' weight (number of "1") which is the result of using dynamic CMOS logic in the design of ARM7TDMI [11]. According to our experiments, we can consider a single weight energy coefficient for all instructions that will cause a negligible error in energy estimation but it will make the model simpler. Therefore, the instruction weight is added as a parameter to our model.

3) *Number of Shift Operation:* To observe the possible effects of the shift operation on the energy consumption of the system, we conducted two different sets of experiments. One group consists of a large number of instructions with shifted operands while the other one contains the same instructions but without using this option. The experiments were repeated for all instructions that can use the shifted operand option and the results show that the shifted version consumes up to 28% more energy than the normal version. It was also observed that the amount of shifted bits only alters the energy consumption about 1%. This behavior is due to the use of barrel shifter [16]. Hence, we only consider the number of shift operations and ignore the number of shifted bits.

4) *Register Bank Bit Flips:* A number of benchmarks based on MVN instruction were prepared to observe the effect of the register bank activity on the system energy consumption. The MVN instruction reads the value of the source operand, performs a bitwise NOT operation and then assigns the result to the destination register. Table I lists the benchmarks used in this experiment.

In all benchmarks, there will be no further activity on the instruction bus or data bus after executing the last MOV instruction. In Benchmark 1, only the first MVN command results in register bank activity and the consequent instructions write the same 0×FFFFFFFF value into R1 register. Benchmark 2 uses the same register for input and output and causes 32 bit flips per cycle in R1 register. Energy measurement of Benchmark 2 shows 3% increase over that of Benchmark 1. To clarify whether the energy consumption increase is related to the Hamming distance or the weight of the registers, we conducted another experiment using Benchmark 3. Benchmark 3 initializes all the registers to value 0×FFFFFFFF in order to increase the total register bank weight to the maximum possible value. Benchmark 1 on the other hand, initializes all the registers to 0 that lowers the total weight of the register bank to the minimum possible value. The energy measurement result of Benchmark 3 shows about 0.1% deviation from the energy consumption of Benchmark 1 that indicates that the register bank energy consumption is more related to the Hamming distance than the weight. Considering the outcome of this experiment, the number of bit flips in the register bank was added as a new parameter to the energy estimation model.

5) *Pipeline Stall:* So far, all parts of (6) are covered with the exception of E_{stall}. There are some instructions that can cause a pipeline stall that will lead to additional energy consumption in the system. These instructions include some types of memory access, multicycle instructions (such as MUL), and the wrong branch prediction. Hence, only the EX stage of the pipeline is active and the other stages are stalled and consume a constant amount of energy.

TABLE I
REGISTER BANK BENCHMARKS

	Benchmark 1	**Benchmark 2**	**Benchmark 3**
	Set all regs to 0	Set all regs to 0	Set all regs to 0×FFFFFFFF
	MOV R2, #0	MOV R1, #0	MOV R2, #0
1	MVN R1, R2	MVN R1, R1	MVN R1, R2
2	MVN R1, R2	MVN R1, R1	MVN R1, R2
...	...	...	...
10000	MVN R1, R2	MVN R1, R1	MVN R1, R2

According to their duration, pipeline stalls can be grouped as follows.

a) The fixed length pipeline stall that delays the execution for a constant amount of cycles. (e.g., the SWP instruction that takes four cycles to execute). The extra energy consumption of these pipeline stalls is added to the base energy cost of the corresponding instructions.
b) The variable length pipeline stall whose duration depends on some items like operand values, previously accessed memory addresses and correct prediction of target of Jump instructions.

Estimating the exact energy consumption of the variable length pipeline stalls needs cycle-accurate simulation of the program. We analyzed the MiBench benchmark suite [8] applications and chose the average energy consumption of the each multicycle instruction as the base energy cost of that instruction. Although this approximation method causes a small error in the energy estimation model, it eliminates the need for cycle-accurate simulation.

C. Static Energy Consumption Model

Since the static energy consumption of the system E_{static} is considered as a constant value, it can be added to the base instruction cost of each instruction type. Therefore, we do not consider a separate parameter for E_{static} in our energy estimation model.

The final energy estimation model can be expressed as

$$E_{\text{app}} = \sum_{p \in P} N_p \times coeff(p) \tag{8}$$

where N_p indicates the total number of occurrence of p during the execution of the application, $coeff(p)$ is the energy coefficient of p and P is the set of {ARM7TDMI instruction set, SRAM read byte ($\lambda_{\text{SRAM(read)}}$ in (5)), SRAM written byte ($\lambda_{\text{SRAM(write)}}$ in (5)), Flash read byte ($\lambda_{\text{Flash(read)}}$ in (5)), register bank bit flip, instruction word Hamming distance, instruction word weight, shift operations}.

V. ANALYSIS

An ARM-based version of SimpleScalar toolset [17] is used to analyze the test programs and extract their energy model parameters. The SimpleScalar toolset contains several simulators with different levels of details and we chose an ISA-level simulator called Sim-profile. A number of modifications have been made to calculate the necessary parameters. Sim-profile analyzes a given ARM binary image and after the simulation, the results are stored on the disk. These reports are further processed to prepare the results for the analysis phase.

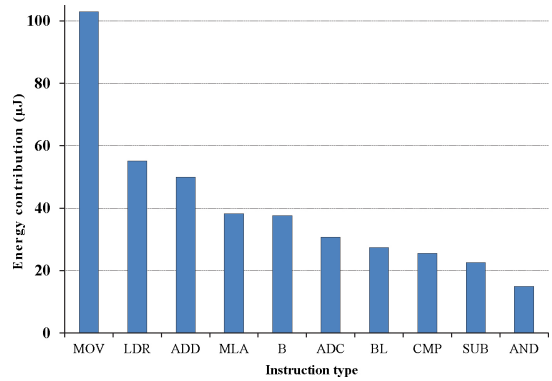

Fig. 4. Top 10 contributor instruction types to total energy consumption of the MiBench benchmark suite applications.

To compute the coefficients of the model, a statistical method known as regression analysis [18] is used. The energy model is assumed to be a linear polynomial function and the factors are extracted using linear regression analysis. In the linear regression analysis the model is in the form of (9) [18]

$$y = c_0 + c_1 x_1 + c_2 x_2 + \ldots + c_n x_n + \epsilon \tag{9}$$

where c_i is the regression coefficient , x_i is the data vector, y is the dependant value and ϵ is the error of approximation. To find the c_i coefficients the least square method [18] was used. Given M independent equations, regression analysis tries to extract the unknown coefficients by assigning different values to them, re-evaluating the polynomials and comparing the result of each polynomial with the corresponding dependant value y. This procedure, known as fitting [18], is repeated for all available equations until either convergence of all the coefficient values or reaching the maximum iteration.

To provide the required equations for the fitting procedure, 60 special test programs were prepared and their energy consumption were measured. Each test program was designed to magnify the effect of one specific model parameter. This improves the accuracy of the fitting procedure by reducing the regression model's complexity. The combination of energy measurement values of these benchmarks and their profile report (analyzed with Sim-profile), form 60 equations for the fitting procedure. Each equation is in the form of (9) where c_i coefficients are $coeff(p)$ coefficients in (8), x_i coefficients are N_p coefficients in (8) and y is the measured energy of the benchmark application, i.e., E_{app} in (8). For example, assume that a benchmark application consists of 5000 occurrence of each of the 38 model parameters (see Table II) and the measured energy consumption of the application is 20 000 μJ, then the corresponding equation is (10)

$$\begin{aligned} 20000\,\mu\text{J} \;=\; & 5000 \times [coeff(ADD) \\ & + coeff(SUB) + \ldots + coeff(SRAM_{\text{stores}})]. \end{aligned} \tag{10}$$

Note that we can make the regression analysis to distribute the static energy consumption over the coeff coefficients.

TABLE II
FINAL RESULTS OF REGRESSION

Instruction parameters									
Param	Energy (nJ)	Param	Energy (nJ)	Param	Energy (nJ)	Param	Energy (nJ)	Param	Energy (nJ)
ADD	0.89	MVN	1.13	ADC	1.127	ORR	1.131	ADDS	1.481
RSB	1.153	AND	1.178	RSC	1.119	B	0.79	SBC	1.113
BIC	1.049	SMLAL	4.391	BL	6.023	SMULL	4.29	CMN	0.976
STM	1.449	CMP	0.978	STR	1.343	EOR	1.167	SUB	1.143
LDM	1.94	SWP	3.593	LDR	1.84	TEQS	1.003	MOV	1.284
TSTS	1.006	MLA	3.423	UMLAL	4.878	MOVS	1.021	UMULL	4.254
MUL	2.931	Shift	0.288						

Interinstruction parameters		Memory parameters	
Parameter	Energy (pJ)	Parameter	Energy (nJ)
Instruction Hamming distance	9.23	Flash loads	2.25
Instruction weight	23.6	SRAM loads	0.72
Regbank bit flip	2.81	SRAM stores	0.98

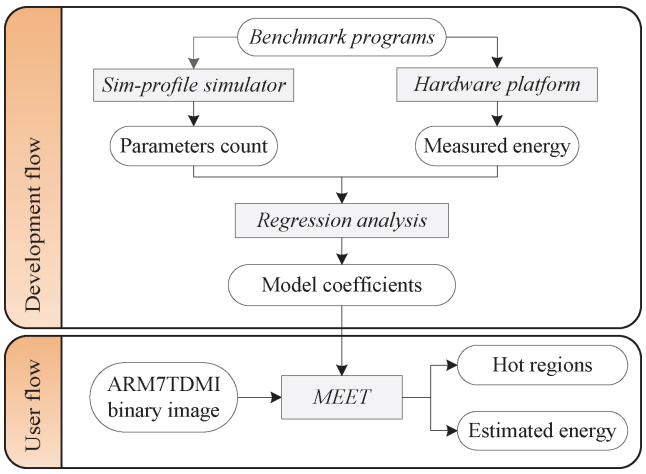

Fig. 5. Energy estimation model derivation process.

Hence, we do not need to consider a constant coefficient in the equation for the static energy.

The ARM7TDMI datasheet indicates that most instructions have two different forms that only one of them updates the flag register. For example ADD and ADDS both perform the addition operation but only ADDS updates the flag registers (e.g., Z flag is set when the result is zero or V flag when an overflow occurs). In the first version of the model, one coefficient was assigned for each one of these pairs. But in order to achieve more accurate results without imposing unnecessary complexity on the model, for some instruction pairs separate coefficients is assigned for each member of the pair. In order to find the most effective instructions in total application energy consumption, the MiBench benchmark suite applications were analyzed with the proposed energy model. The top 10 contributors are listed in Fig. 4. According to the results, the two versions of ADD and MOV instructions were separated, which improved the estimation accuracy of the model and confirmed the validity of our decision.

Fig. 5 summarizes the approach that we have followed in this paper to develop the energy estimation model. It also illustrates the flow that the user of our tool must follow to estimate the energy consumption of the application and identify the hotspots of the source code.

TABLE III
ENERGY ESTIMATION OF MIBENCH

Application	Estimated (J)	Actual (J)	Error (%)
Basicmath	1.1713	1.2066	−2.93
Bitcnts	0.5758	0.6011	−4.22
CRC32	0.036	0.035	+2.73
FFT	0.2243	0.2268	−1.11
Quicksort	0.0285	0.0304	−5.97
SHA	0.701	0.6627	+5.77
Stringsearch	0.423	0.4105	+3.03

VI. RESULTS

The final results of the regression analysis is summarized in Table II. To measure the quality of the regression we used a criterion named coefficient of determination, which is denoted by R^2 [18]. The reported coefficient of determination for our final regression analysis is 0.9987, which means 99.87% of all variations of the test applications energy consumption are captured by the proposed energy estimation model.

In order to test the accuracy of the model in real world applications, a set of experiments were conducted on MiBench benchmark suite applications. Table III shows that the energy consumption of all applications are estimated with less than 6% error. Some sources of error in our energy estimation model are ignoring the Hamming distance and weight of address bus and data bus and using a constant value to model the energy consumption of variable length pipeline stalls to avoid cycle-accurate simulation of the system (see Section IV-B5). Considering the value of R^2 it is expected to achieve better accuracy, which means the test programs can be improved to cover all aspects of microcontroller energy consumption.

Having the final regression results, the energy coefficients were embedded in the Sim-profile simulator from SimpleScalar toolset to make the process of energy estimation easier. Since Sim-profile has the ability to profile the application based on a given simulation parameter, adding the energy consumption estimation ability makes it an energy profiler tool. This profiler tool is called MEET. MEET can be used for finding blocks of software with high energy consumption. It can also provide useful guidelines about optimizing the

energy consumption by reporting the cause of high energy consumption in hot regions.

VII. Conclusion and Future Work

An instruction-level energy estimation model for a microcontroller-based embedded system was presented. By combining similar energy coefficient values, the presented model became much simpler than most proposed models that can speed up the energy estimation process and therefore the development of the embedded systems. The model was validated against a physical hardware platform, and the error of estimation for a number of applications from the MiBench benchmark suite was less than 6%.

Retargetability is another advantage of our proposed model that makes it easy to adjust the model coefficients for a new platform. The model coefficients can be calibrated by measuring the energy consumption of test programs for the new platform and rerunning the regression analysis.

Also, a tool called MEET was developed, which receives a binary ARM7 application and profiles the energy consumption of the application on an AT91SAM7x256 microcontroller. The results can identify the hotspots of the code and help in selecting the best coding technique in order to optimize the total energy consumption of the application.

Considering that an estimation tool with the capability of analyzing the energy consumption of peripherals can also be very helpful, we think the energy estimation of peripherals is a possible candidate for our future research works.

References

[1] V. Tiwari, S. Malik, and A. Wolfe, "Power analysis of embedded software: A first step towards software power minimization," *IEEE Trans. VLSI Systems*, vol. 2, no. 4, pp. 437–445, Dec. 1994.

[2] B. Klass, D. E. Thomas, H. Schmit, and D. F. Nagle, "Modeling inter-instruction energy effects in a digital signal processor," in *Proc. Digital Signal Processor, Power-Driven Microarch. Workshop in Conjunction with Int. Symp. Comput. Arch.*, Jun. 1998, pp. 18–23.

[3] G. Callou, P. Maciel, E. Tavares, E. Andrade, B. Nogueira, C. Araujo, and P. Cunha, "Energy consumption and execution time estimation of embedded system applications," *Microprocessors Microsyst.*, vol. 35, no. 4, pp. 426–440, 2011.

[4] N. Chang, K. Kim, and H. G. Lee, "Cycle-accurate energy consumption measurement and analysis: Case study of ARM7TDMI," in *Proc. ISLPED*, 2000, pp. 185–190.

[5] S. Nikolaidis, N. Kavvadias, T. Laopoulos, L. Bisdounis, and S. Blionas, "Instruction level energy modeling for pipelined processors," *J. Embedding Comput.*, vol. 1, no. 3, pp. 317–324, Aug. 2005.

[6] V. Konstantakos, A. Chatzigeorgiou, S. Nikolaidis, and T. Laopoulos, "Energy consumption estimation in embedded systems," *IEEE Trans. Instrum. Meas.*, vol. 57, no. 4, pp. 797–804, Apr. 2008.

[7] S. Lee, A. Ermedahl, and S. L. Min, "An accurate instruction-level energy consumption model for embedded RISC processors," in *Proc. 5th ACM SIGPLAN Workshop LCTES*, Aug. 2001, pp. 1–10.

[8] M. R. Guthaus, J. S. Ringenberg, D. Ernst, T. M. Austin, T. Mudge, and R. B. Brown, "MiBench: A free, commercially representative embedded benchmark suite," in *Proc. IEEE Int. Workshop Workload Characterization*, Dec. 2001, pp. 3–14.

[9] N. Kavvadias, P. Neofotistos, S. Nikolaidis, K. Kosmatopoulos, and T. Laopoulos, "Measurements analysis of the software-related power consumption of microprocessors," *IEEE Trans. Instrum. Measurement*, vol. 53, no. 4, pp. 1106–1112, Aug. 2004.

[10] K. Zotos, A. Litke, E. Chatzigeorgiou, S. Nikolaidis, and G. Stephanides, "Energy complexity of software in embedded systems," in *Proc. IASTED*, Jun. 2005, pp. 17–27.

[11] S. Steinke, M. Knauer, L. Wehmeyer, and P. Marwedel, "An accurate and fine grain instruction-level energy model supporting software optimizations," in *Proc. Int. Workshop PATMOS*, Sep. 2001.

[12] V. Konstantakos, A. Chatzigeorgiou, S. Nikolaidis, and T. Laopoulos, "Energy consumption estimation in embedded systems," in *Proc. IEEE IMTC*, Apr. 2006, pp. 235–238.

[13] M. Lee, V. Tiwari, S. Malik, and M. Fujita, "Power analysis and minimization techniques for embedded DSP software," *IEEE Trans. VLSI Syst.*, vol. 5, no. 1, pp. 123–135, Mar. 1997.

[14] J. T. Russell and M. F. Jacome, "Software power estimation and optimization for high performance, 32-bit embedded processors," in *Proc. ICCD: VLSI Comput. Processors*, Oct. 1998, pp. 328–333.

[15] *AT91SAM7X256 Data Sheet*, Atmel, 2009.

[16] *ARM7TDMI Data Sheet*, Advanced RISC Machines Ltd (ARM), 1995.

[17] D. Burger and T. M. Austin, "The simplescalar tool set, version 2.0," *SIGARCH Comput. Archit. News*, vol. 25, no. 3, pp. 13–25, 1997.

[18] S. Chatterjee and A. S. Hadi, *Regression Analysis by Example*. 4th ed., New York, USA: Wiley, 2006.

Mostafa Bazzaz is currently pursuing the Ph.D. degree at the Computer Engineering Department, Sharif University of Technology, Tehran, Iran.

His current research interests include low-power multicore embedded systems, software-based fault tolerance, and real-time embedded systems.

Mohammad Salehi is currently pursuing the Ph.D. degree in computer engineering at the Sharif University of Technology, Tehran, Iran.

His current research interests include embedded systems, low-power design, and the tradeoff between fault tolerance and energy efficiency in real-time systems.

Alireza Ejlali is an Associate Professor of computer engineering with the Sharif University of Technology. He is currently the Director of the Computer Architecture Group and the Embedded Systems Research Laboratory, Department of Computer Engineering, Sharif University of Technology. His current research interests include low power design, real-time embedded systems, and fault-tolerant embedded systems.